

BÁO CÁO THIẾT KẾ HỆ THỐNG: RECYCLING LAB TYCOON (AI-POWERED GAME)

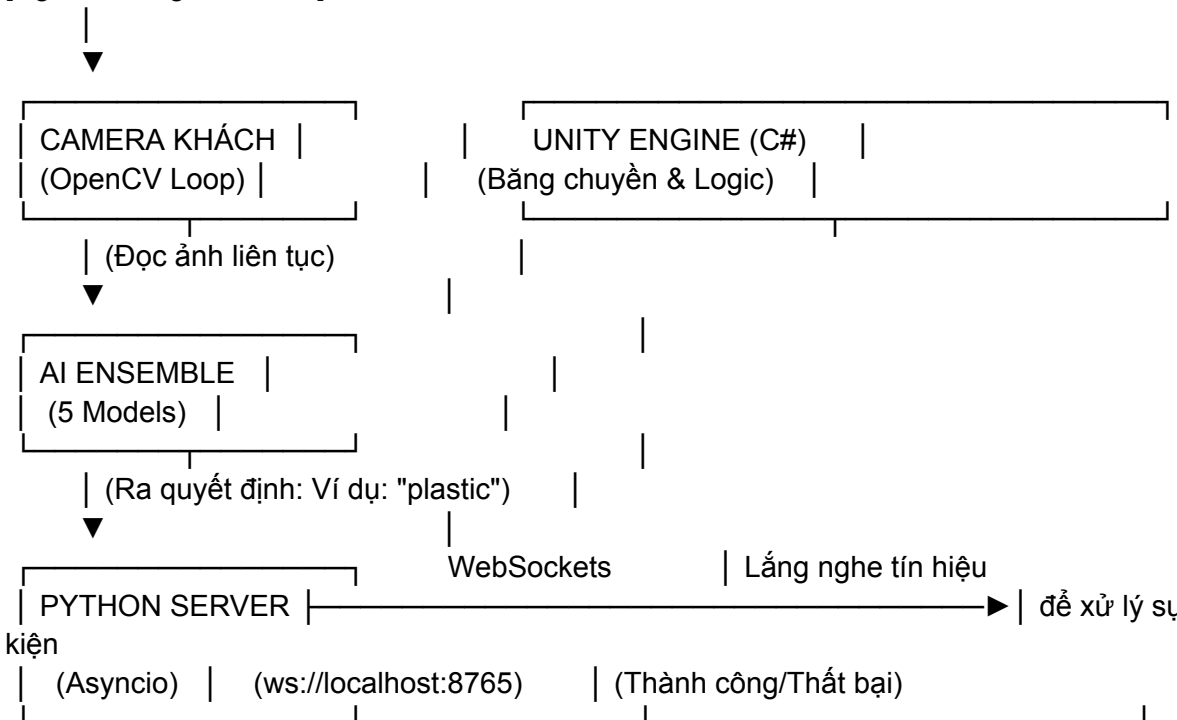
1. TỔNG QUAN DỰ ÁN (PROJECT OVERVIEW)

- Tên dự án:** Recycling Lab Tycoon
- Thể loại:** Mô phỏng, Quản lý (Tycoon / Simulation) kèm cơ chế Gamification.
- Điểm độc đáo (Core Feature):** Người chơi tương tác với thế giới ảo trong game bằng cách **giơ các loại rác thải thật ngoài đời trước Camera/Webcam**. Hệ thống AI tích hợp sẽ tự động nhận diện và kích hoạt logic game tương ứng.

2. KIẾN TRÚC LUỒNG DỮ LIỆU TỔNG THỂ (SYSTEM ARCHITECTURE)

Hệ thống hoạt động theo mô hình **Client - Server** thời gian thực, tách biệt hoàn toàn giữa giao diện trò chơi (Frontend) và lõi xử lý trí tuệ nhân tạo (Backend).

[Người chơi giơ rác thật]



3. THIẾT KẾ CHI TIẾT CHO PHÒNG BAN AI (BACKEND DEVELOPMENT)

3.1. Nhiệm vụ cốt lõi

Xây dựng một WebSocket Server bằng Python đóng vai trò xử lý luồng hình ảnh từ Camera, chạy suy luận (Inference) song song trên 5 mô hình Học sâu và trả ra kết quả đồng thuận tốt nhất.

3.2. Cấu trúc 5 Mô hình mạng (Ensemble Models)

Hệ thống sử dụng chiến lược **Tập hợp biểu quyết (Voting Ensemble)** từ 5 kiến trúc phân loại ảnh (chuẩn hóa đầu ra **12 lớp rác thải**):

1. **ResNet50**: Kiến trúc nền tảng (Baseline), trích xuất đặc trưng mạnh mẽ.
2. **EfficientNet-B0**: Mô hình tối ưu hiệu năng, độ chính xác cao.
3. **EfficientNet-V2-S**: Cải tiến tốc độ huấn luyện và độ chuẩn xác biên độ lớn.
4. **MobileNetV3-Large**: Kiến trúc siêu nhẹ, đảm bảo tốc độ phản hồi nhanh.
5. **ConvNeXt-Tiny**: Kiến trúc dạng CNN hiện đại nâng cao, đạt điểm số tối ưu trên tập kiểm thử.

3.3. Thuật toán Biểu quyết (Majority Voting Algorithm)

- Mỗi mô hình đóng góp 1 lá phiếu dự đoán (12 lớp rác).
- **Kịch bản 1 (Đồng thuận)**: Class nào có số phiếu cao nhất (≥ 2 phiếu và không bị đồng hạng) → Lấy Class đó làm kết quả cuối cùng.
- **Kịch bản 2 (Bất đồng/Hòa phiếu)**: Nếu mỗi mô hình ra một kết quả khác nhau (ví dụ: 5 dòng rác khác nhau), thuật toán sẽ chọn Class của mô hình có **Độ tự tin (Confidence Score / Softmax Probability)** cao nhất làm phương án dự phòng (Fallback).

3.4. Định dạng dữ liệu đầu ra (JSON Packet qua WebSocket)

Mỗi chu kỳ 100ms (10 FPS), Server sẽ gửi về Unity một chuỗi JSON có cấu trúc sau:

```
JSON
{
  "detected_item": "plastic",
  "confidence": 0.89,
  "all_votes": ["plastic", "plastic", "cardboard", "plastic", "glass"]
}
```

4. THIẾT KẾ CHI TIẾT CHO PHÒNG BAN GAME (UNITY FRONTEND)

4.1. Nhiệm vụ cốt lõi

Xây dựng thế giới game 2D/3D điều khiển bằng chuyền rác, quản lý trạng thái kinh tế (tiền vàng), kết nối và xử lý dữ liệu bất đồng bộ nhận được từ cổng WebSocket.

4.2. Logic trò chơi (Core Game Loop)

1. **Hệ thống Băng chuyền**: Các vật thể rác ảo (Vỏ chai, Cục pin, Thùng carton) sẽ liên tục được sinh ra và di chuyển trên băng chuyền đến "Vùng Tái Chế".

2. **Mục tiêu hiện tại (currentRequiredTrash):** Game hiển thị loại rác đang cần phân loại ở đầu hàng (Ví dụ: Game hiển thị biểu tượng "Nhựa").
3. **Điều kiện Kích hoạt (Match & Trigger):** * Unity liên tục nhận gói tin JSON từ WebSocket.
 - Nếu **detected_item** (AI trả về) **TRÙNG** với **currentRequiredTrash** (Game đang yêu cầu) **VÀ** **confidence** > 0.65 (Ngăn chặn việc gian lận hoặc nhiễu camera).
 - $\rightarrow$ Kích hoạt trạng thái **Tái chế thành công**: Phát hiệu ứng (VFX), cộng tiền vàng (**goldCount** $+= 100$), đẩy vật thể rác vào lò tái chế ảo và đổi mục tiêu tiếp theo trên băng chuyền.

4.3. Quản lý trạng thái và UI (User Interface)

- **UI Bảng điều khiển (Dashboard):** Hiển thị luồng thông tin AI đang "nghĩ" (Có thể hiển thị danh sách **all_votes** dưới dạng các biểu tượng Robot nhỏ đang tranh luận để tăng tính trực quan và vui nhộn cho Game).
- **Cơ chế làm mượt (Debounce/Chống nhiễu):** Tránh việc cộng tiền trùng lặp nhiều lần cho một vật thể. Khi một vật thể đã được xác nhận tái chế thành công, Unity phải tạm khóa (Lock) việc nhận tín hiệu loại rác đó trong vòng \$1\$ - \$2\$ giây cho đến khi vật thể tiếp theo tiến vào vùng quét.

5. KẾ HOẠCH TRIỂN KHAI PHỐI HỢP (DEPLOYMENT PLAN)

1. **Giai đoạn 1 (AI Team):** Hoàn thiện huấn luyện 5 mô hình trên Kaggle $\rightarrow$ Xuất 5 file **.pth** $\rightarrow$ Viết Script chạy Local WebSocket Server ổn định.
2. **Giai đoạn 2 (Unity Team):** Thiết kế đồ họa băng chuyền, import Package **NativeWebSocket** để dựng sẵn khung nhận dữ liệu mẫu (Mock Data).
3. **Giai đoạn 3 (Tích hợp - Integration):** Kết nối trực tiếp hai hệ thống trên cùng một máy nội bộ (**localhost**). Kiểm thử thực tế với vật thể thật để căn chỉnh ngưỡng độ tin tưởng (**confidence**) tối ưu nhất cho Camera.